

Java Code

```
1  import java.sql.*;
2  import java.util.Scanner;
3
4  public class jdbcproject {
5
6      static Scanner sc = new Scanner(System.in);
7
8      public static void main(String[] args) {
9          String url      = "jdbc:mysql://localhost:3306/University";
10         String user      = "root";
11         String password = "170305";
12
13         try (Connection conn = DriverManager.getConnection(url, user, password)) {
14
15             if (conn != null) {
16                 System.out.println("Connected to the database!");
17             }
18
19             int ch;
20             do {
21                 System.out.println("\n\n***** University Management System *****");
22                 System.out.println("1.  Show Student Records");
23                 System.out.println("2.  Add Student Record");
24                 System.out.println("3.  Delete Student Record");
25                 System.out.println("4.  Update Student Information");
26                 System.out.println("5.  Show Instructor Details");
27                 System.out.println("6.  Show Course Details with Enrolled Students");
28                 System.out.println("7.  Show Course Details taken by Instructor");
29                 System.out.println("8.  Deposit HRA To Salary");
30                 System.out.println("9.  Deduct TDS From Salary");
31                 System.out.println("10. EXIT The Program");
32                 System.out.print("Enter your choice (1-10): ");
33                 ch = sc.nextInt();
34
35                 switch (ch) {
36                     case 1:
37                         showStudentRecords(conn);
38                         break;
39                     case 2:
40                         addStudentRecord(conn);
41                         showStudentRecords(conn);
42                         break;
43                     case 3:
44                         deleteStudentRecord(conn);
45                         showStudentRecords(conn);
46                         break;
47                     case 4:
48                         updateStudentRecord(conn);
49                         showStudentRecords(conn);
50                         break;
```

```

51         case 5:
52             showInstructorDetails(conn);
53             break;
54         case 6:
55             showCourseWithEnrolledStudents(conn);
56             break;
57
58         case 7:
59             showCoursesByInstructor(conn);
60             break;
61         case 8:
62             depositHRA(conn);
63             break;
64         case 9:
65             deductTDS(conn);
66             break;
67         case 10:
68             System.out.println("Exiting... Goodbye!");
69             break;
70         default:
71             System.out.println("Invalid choice! Please enter a value between 1 and
10.");
72     }
73
74     } while (ch != 10);
75
76     } catch (SQLException e) {
77         System.out.println("Connection failed: " + e.getMessage());
78     }
79 }
80
81 // CASE 1 - Show all student records
82 static void showStudentRecords(Connection conn) {
83     String query = "SELECT * FROM student ORDER BY ID";
84     try (Statement stmt = conn.createStatement();
85         ResultSet rs = stmt.executeQuery(query)) {
86
87         System.out.println("\n" + "=".repeat(65));
88         System.out.printf("%-10s %-20s %-15s %-10s\n",
89             "ID", "Name", "Dept_Name", "Tot_Cred");
90         System.out.println("-".repeat(65));
91
92         boolean found = false;
93         while (rs.next()) {
94             found = true;
95             System.out.printf("%-10s %-20s %-15s %-10d\n",
96                 rs.getString("ID"),
97                 rs.getString("name"),
98                 rs.getString("dept_name"),
99                 rs.getInt("tot_cred"));
100         }
101         if (!found) {
102             System.out.println("No student records found.");
103         }

```

```

104         System.out.println("=".repeat(65));
105
106     } catch (SQLException e) {
107         System.out.println("Error fetching student records: " + e.getMessage());
108     }
109 }
110
111 // CASE 2 - Add a new student
112 static void addStudentRecord(Connection conn) {
113     System.out.print("Enter Student ID   : ");
114     String id = sc.next();
115     sc.nextLine(); // flush
116     System.out.print("Enter Student Name : ");
117     String name = sc.nextLine();
118     System.out.print("Enter Department   : ");
119     String dept = sc.nextLine();
120     System.out.print("Enter Total Credits : ");
121     int cred = sc.nextInt();
122
123     String sql = "INSERT INTO student (ID, name, dept_name, tot_cred) VALUES (?, ?, ?,
124 ?)";
125     try (PreparedStatement ps = conn.prepareStatement(sql)) {
126         ps.setString(1, id);
127         ps.setString(2, name);
128         ps.setString(3, dept);
129         ps.setInt(4, cred);
130         int rows = ps.executeUpdate();
131         if (rows > 0) {
132             System.out.println("Student record added successfully!");
133         }
134     } catch (SQLIntegrityConstraintViolationException e) {
135         System.out.println("Error: Student with ID " + id + " already exists.");
136     } catch (SQLException e) {
137         System.out.println("Error adding student: " + e.getMessage());
138     }
139
140 // CASE 3 - Delete a student
141 static void deleteStudentRecord(Connection conn) {
142     System.out.print("Enter Student ID to delete: ");
143     String id = sc.next();
144
145     if (studentExists(conn, id)) {
146         System.out.println("No student found with ID " + id + ".");
147         return;
148     }
149
150     try {
151         // Step 1: Delete from takes (child table) first
152         String deleteTakes = "DELETE FROM takes WHERE ID = ?";
153         try (PreparedStatement ps = conn.prepareStatement(deleteTakes)) {
154             ps.setString(1, id);
155             int rows = ps.executeUpdate();
156             if (rows > 0) {

```

```

157         System.out.println("Removed " + rows + " course enrollment(s) for student "
+ id + ".");
158     }
159 }
160
161 // Step 2: Now delete from student (parent table)
162 String deleteStudent = "DELETE FROM student WHERE ID = ?";
163 try (PreparedStatement ps = conn.prepareStatement(deleteStudent)) {
164     ps.setString(1, id);
165     ps.executeUpdate();
166     System.out.println("Student record with ID " + id + " deleted successfully.");
167 }
168
169 } catch (SQLException e) {
170     System.out.println("Error deleting student: " + e.getMessage());
171 }
172 }
173
174 // CASE 4 - Update a student's record
175 static void updateStudentRecord(Connection conn) {
176     System.out.print("Enter Student ID to update: ");
177     String id = sc.next();
178
179     // Verify student exists
180     if (studentExists(conn, id)) {
181         System.out.println("No student found with ID " + id + ".");
182         return;
183     }
184
185     System.out.println("What do you want to update?");
186     System.out.println("  1: Name");
187     System.out.println("  2: Dept_Name");
188     System.out.println("  3: Tot_Cred");
189     System.out.print("Enter choice (1/2/3): ");
190     int ch1 = sc.nextInt();
191
192     String sql = "";
193     switch (ch1) {
194         case 1:
195             sc.nextLine();
196             System.out.print("Enter new Name: ");
197             String newName = sc.nextLine();
198             sql = "UPDATE student SET name = ? WHERE ID = ?";
199             executeUpdate(conn, sql, newName, id);
200             break;
201
202         case 2:
203             sc.nextLine();
204             System.out.print("Enter new Department: ");
205             String newDept = sc.nextLine();
206             sql = "UPDATE student SET dept_name = ? WHERE ID = ?";
207             executeUpdate(conn, sql, newDept, id);
208             break;
209

```

```

210         case 3:
211             System.out.print("Enter new Total Credits: ");
212             int newCred = sc.nextInt();
213             sql = "UPDATE student SET tot_cred = ? WHERE ID = ?";
214             executeUpdate(conn, sql, String.valueOf(newCred), id);
215             break;
216
217         default:
218             System.out.println("Invalid choice! Please enter 1, 2, or 3.");
219     }
220 }
221
222 /** Helper - executes a 2-param UPDATE (value, id) and prints outcome. */
223 static void executeUpdate(Connection conn, String sql, String value, String id) {
224     try (PreparedStatement ps = conn.prepareStatement(sql)) {
225         ps.setString(1, value);
226         ps.setString(2, id);
227         int rows = ps.executeUpdate();
228         if (rows > 0) {
229             System.out.println("Record updated successfully.");
230         } else {
231             System.out.println("No record updated.");
232         }
233     } catch (SQLException e) {
234         System.out.println("Error updating student: " + e.getMessage());
235     }
236 }
237
238 // CASE 5 - Show instructor details
239 static void showInstructorDetails(Connection conn) {
240     System.out.print("Enter Instructor ID: ");
241     String id = sc.next();
242
243     String sql = "SELECT * FROM instructor WHERE ID = ?";
244     try (PreparedStatement ps = conn.prepareStatement(sql)) {
245         ps.setString(1, id);
246         ResultSet rs = ps.executeQuery();
247
248         System.out.println("\n" + "=".repeat(65));
249         System.out.printf("%-10s %-20s %-15s %-10s\n",
250             "ID", "Name", "Dept_Name", "Salary");
251         System.out.println("-".repeat(65));
252
253         if (rs.next()) {
254             System.out.printf("%-10s %-20s %-15s %-10.2f\n",
255                 rs.getString("ID"),
256                 rs.getString("name"),
257                 rs.getString("dept_name"),
258                 rs.getDouble("salary"));
259         } else {
260             System.out.println("No instructor found with ID " + id + ".");
261         }
262         System.out.println("=".repeat(65));
263     }

```

```

264     } catch (SQLException e) {
265         System.out.println("Error fetching instructor: " + e.getMessage());
266     }
267 }
268
269 // CASE 6 - Show course details with enrolled students
270 static void showCourseWithEnrolledStudents(Connection conn) {
271     System.out.print("Enter Course ID: ");
272     String courseId = sc.next();
273
274     // Show course info
275     String courseSql = "SELECT * FROM course WHERE course_id = ?";
276     try (PreparedStatement ps = conn.prepareStatement(courseSql)) {
277         ps.setString(1, courseId);
278         ResultSet rs = ps.executeQuery();
279
280         System.out.println("\n" + "=".repeat(70));
281         if (rs.next()) {
282             System.out.println("Course ID      : " + rs.getString("course_id"));
283             System.out.println("Course Name   : " + rs.getString("title"));
284             System.out.println("Department    : " + rs.getString("dept_name"));
285             System.out.println("Credits       : " + rs.getInt("credits"));
286         } else {
287             System.out.println("No course found with ID " + courseId + ".");
288             System.out.println("=".repeat(70));
289             return;
290         }
291     } catch (SQLException e) {
292         System.out.println("Error fetching course: " + e.getMessage());
293         return;
294     }
295
296     // Show enrolled students (via 'takes' table)
297     String studentSql =
298         "SELECT s.ID, s.name, t.semester, t.year " +
299         "FROM student s JOIN takes t ON s.ID = t.ID " +
300         "WHERE t.course_id = ? " +
301         "ORDER BY t.year, t.semester";
302
303     try (PreparedStatement ps = conn.prepareStatement(studentSql)) {
304         ps.setString(1, courseId);
305         ResultSet rs = ps.executeQuery();
306
307         System.out.println("\nEnrolled Students:");
308         System.out.println("-".repeat(70));
309         System.out.printf("%-10s %-25s %-12s %-6s\n",
310             "Stu ID", "Name", "Semester", "Year");
311         System.out.println("-".repeat(70));
312
313         boolean found = false;
314         while (rs.next()) {
315             found = true;
316             System.out.printf("%-10s %-25s %-12s %-6d\n",
317                 rs.getString("ID"),

```

```

318         rs.getString("name"),
319         rs.getString("semester"),
320         rs.getInt("year"));
321     }
322     if (!found) {
323         System.out.println("No students enrolled in this course.");
324     }
325     System.out.println("=".repeat(70));
326
327 } catch (SQLException e) {
328     System.out.println("Error fetching enrolled students: " + e.getMessage());
329 }
330 }
331
332 // CASE 7 - Show courses taught by an instructor
333 static void showCoursesByInstructor(Connection conn) {
334     System.out.print("Enter Instructor ID: ");
335     String id = sc.next();
336
337     String sql =
338         "SELECT i.ID AS instr_id, i.name AS instr_name, " +
339         "        c.course_id, c.title, c.dept_name, c.credits, " +
340         "        t.semester, t.year " +
341         "FROM instructor i " +
342         "JOIN teaches t ON i.ID = t.ID " +
343         "JOIN course c ON t.course_id = c.course_id " +
344         "WHERE i.ID = ? " +
345         "ORDER BY t.year, t.semester";
346
347     try (PreparedStatement ps = conn.prepareStatement(sql)) {
348         ps.setString(1, id);
349         ResultSet rs = ps.executeQuery();
350
351         System.out.println("\n" + "=".repeat(80));
352         System.out.printf("%-10s %-18s %-10s %-22s %-12s %-6s%n",
353             "Instr ID", "Instr Name", "Course ID",
354             "Title", "Semester", "Year");
355         System.out.println("-".repeat(80));
356
357         boolean found = false;
358         while (rs.next()) {
359             found = true;
360             System.out.printf("%-10s %-18s %-10s %-22s %-12s %-6d%n",
361                 rs.getString("instr_id"),
362                 rs.getString("instr_name"),
363                 rs.getString("course_id"),
364                 rs.getString("title"),
365                 rs.getString("semester"),
366                 rs.getInt("year"));
367         }
368         if (!found) {
369             System.out.println("No courses found for Instructor ID " + id + ".");
370         }
371         System.out.println("=".repeat(80));

```

```

372
373     } catch (SQLException e) {
374         System.out.println("Error fetching courses: " + e.getMessage());
375     }
376 }
377
378 // CASE 8 - Deposit HRA (15% of salary)
379 static void depositHRA(Connection conn) {
380     System.out.print("Enter Instructor ID: ");
381     String id = sc.next();
382
383     if (instructorExists(conn, id)) {
384         System.out.println("No instructor found with ID " + id + ".");
385         return;
386     }
387
388     String updateSql = "UPDATE instructor SET salary = salary + (salary * 0.15) WHERE
ID = ?";
389     try (PreparedStatement ps = conn.prepareStatement(updateSql)) {
390         ps.setString(1, id);
391         ps.executeUpdate();
392         System.out.println("HRA (15%) deposited successfully to Instructor " + id +
".");
393     } catch (SQLException e) {
394         System.out.println("Error depositing HRA: " + e.getMessage());
395         return;
396     }
397
398     System.out.println("Updated Instructor Details:");
399     showInstructorDetails(conn);
400 }
401
402 // CASE 9 - Deduct TDS (20% of salary)
403 static void deductTDS(Connection conn) {
404     System.out.print("Enter Instructor ID: ");
405     String id = sc.next();
406
407     if (instructorExists(conn, id)) {
408         System.out.println("No instructor found with ID " + id + ".");
409         return;
410     }
411
412     // Get current salary first
413     double currentSalary = 0;
414     String selectSql = "SELECT salary FROM instructor WHERE ID = ?";
415     try (PreparedStatement ps = conn.prepareStatement(selectSql)) {
416         ps.setString(1, id);
417         ResultSet rs = ps.executeQuery();
418         if (rs.next()) {
419             currentSalary = rs.getDouble("salary");
420         }
421     } catch (SQLException e) {
422         System.out.println("Error reading salary: " + e.getMessage());
423         return;

```



```

424     }
425
426     double tds = currentSalary * 0.20;
427     double newSalary = currentSalary - tds;
428
429     if (newSalary < 0) {
430         System.out.println("Error: TDS deduction would result in negative salary.
Transaction aborted.");
431         return;
432     }
433
434     String updateSql = "UPDATE instructor SET salary = salary - (salary * 0.20) WHERE
ID = ?";
435     try (PreparedStatement ps = conn.prepareStatement(updateSql)) {
436         ps.setString(1, id);
437         ps.executeUpdate();
438         System.out.printf("TDS (20%%) of %.2f deducted successfully from Instructor
%s.%n", tds, id);
439     } catch (SQLException e) {
440         System.out.println("Error deducting TDS: " + e.getMessage());
441         return;
442     }
443
444     System.out.println("Updated Instructor Details:");
445     showInstructorDetails(conn);
446 }
447
448 // Helpers - existence checks
449 static boolean studentExists(Connection conn, String id) {
450     String sql = "SELECT 1 FROM student WHERE ID = ?";
451     try (PreparedStatement ps = conn.prepareStatement(sql)) {
452         ps.setString(1, id);
453         ResultSet rs = ps.executeQuery();
454         return !rs.next();
455     } catch (SQLException e) {
456         return true;
457     }
458 }
459
460 static boolean instructorExists(Connection conn, String id) {
461     String sql = "SELECT 1 FROM instructor WHERE ID = ?";
462     try (PreparedStatement ps = conn.prepareStatement(sql)) {
463         ps.setString(1, id);
464         ResultSet rs = ps.executeQuery();
465         return !rs.next();
466     } catch (SQLException e) {
467         return true;
468     }
469 }
470 }

```

```
"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ
IDEA Community Edition 2025.2.6.1\lib\idea_rt.jar=63034" -Dfile.encoding=UTF-8 -
Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath
"C:\Users\puspa\Downloads\IDB Project
Solution\IDB_Project\out\production\IDB_Project;C:\Users\puspa\Downloads\IDB Project
Solution\IDB_Project\src\mysql-connector-j-9.4.0.jar" jdbcproject
```

Connected to the database!

***** University Management System *****

1. Show Student Records
2. Add Student Record
3. Delete Student Record
4. Update Student Information
5. Show Instructor Details
6. Show Course Details with Enrolled Students
7. Show Course Details taken by Instructor
8. Deposit HRA To Salary
9. Deduct TDS From Salary
10. EXIT The Program

Enter your choice (1-10): 1

=====

ID	Name	Dept_Name	Tot_Cred
----	------	-----------	----------

00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54

55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

=====

***** University Management System *****

1. Show Student Records
2. Add Student Record
3. Delete Student Record
4. Update Student Information
5. Show Instructor Details
6. Show Course Details with Enrolled Students
7. Show Course Details taken by Instructor
8. Deposit HRA To Salary
9. Deduct TDS From Salary
10. EXIT The Program

Enter your choice (1-10): 2

Enter Student ID : 23410

Enter Student Name : Puspai

Enter Department : Comp. Sci.

Enter Total Credits : 200

Student record added successfully!

=====

ID	Name	Dept_Name	Tot_Cred
----	------	-----------	----------

00128	Zhang	Comp. Sci.	102
-------	-------	------------	-----

12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
23410	Puspal	Comp. Sci.	200
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

=====

***** University Management System *****

1. Show Student Records
2. Add Student Record
3. Delete Student Record
4. Update Student Information
5. Show Instructor Details
6. Show Course Details with Enrolled Students
7. Show Course Details taken by Instructor
8. Deposit HRA To Salary
9. Deduct TDS From Salary
10. EXIT The Program

Enter your choice (1-10): 3

Enter Student ID to delete: 98988

Error deleting student: Unknown column 'ID' in 'where clause'

=====

ID	Name	Dept_Name	Tot_Cred
----	------	-----------	----------

00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
23410	Puspal	Comp. Sci.	200
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

=====

***** University Management System *****

1. Show Student Records
2. Add Student Record
3. Delete Student Record
4. Update Student Information
5. Show Instructor Details
6. Show Course Details with Enrolled Students
7. Show Course Details taken by Instructor
8. Deposit HRA To Salary
9. Deduct TDS From Salary

10. EXIT The Program

Enter your choice (1-10): 4

Enter Student ID to update: 23410

What do you want to update?

1: Name

2: Dept_Name

3: Tot_Cred

Enter choice (1/2/3): 3

Enter new Total Credits: 180

Record updated successfully.

```
=====
```

ID	Name	Dept_Name	Tot_Cred

00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
23410	Puspal	Comp. Sci.	180
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

```
=====
```

***** University Management System *****

1. Show Student Records
2. Add Student Record
3. Delete Student Record
4. Update Student Information
5. Show Instructor Details
6. Show Course Details with Enrolled Students
7. Show Course Details taken by Instructor
8. Deposit HRA To Salary
9. Deduct TDS From Salary
10. EXIT The Program

Enter your choice (1-10): 5

Enter Instructor ID: 12121

=====

ID	Name	Dept_Name	Salary
----	------	-----------	--------

12121	Wu	Finance	90000.00
-------	----	---------	----------

=====

***** University Management System *****

1. Show Student Records
2. Add Student Record
3. Delete Student Record
4. Update Student Information
5. Show Instructor Details
6. Show Course Details with Enrolled Students
7. Show Course Details taken by Instructor
8. Deposit HRA To Salary
9. Deduct TDS From Salary

10. EXIT The Program

Enter your choice (1-10): 6

Enter Course ID: BIO-101

=====

Course ID : BIO-101

Course Name : Intro. to Biology

Department : Biology

Error fetching course: Column 'credits' not found.

***** University Management System *****

1. Show Student Records
2. Add Student Record
3. Delete Student Record
4. Update Student Information
5. Show Instructor Details
6. Show Course Details with Enrolled Students
7. Show Course Details taken by Instructor
8. Deposit HRA To Salary
9. Deduct TDS From Salary
10. EXIT The Program

Enter your choice (1-10): 8

Enter Instructor ID: 10101

HRA (15%) deposited successfully to Instructor 10101.

Updated Instructor Details:

Enter Instructor ID: 12121

=====

ID	Name	Dept_Name	Salary

12121	Wu	Finance	90000.00

***** University Management System *****

1. Show Student Records
2. Add Student Record
3. Delete Student Record
4. Update Student Information
5. Show Instructor Details
6. Show Course Details with Enrolled Students
7. Show Course Details taken by Instructor
8. Deposit HRA To Salary
9. Deduct TDS From Salary
10. EXIT The Program

Enter your choice (1-10): 9

Enter Instructor ID: 12121

TDS (20%) of 18000.00 deducted successfully from Instructor 12121.

Updated Instructor Details:

Enter Instructor ID: 98345

ID	Name	Dept_Name	Salary
----	------	-----------	--------

98345	Kim	Elec. Eng.	80000.00
-------	-----	------------	----------

***** University Management System *****

1. Show Student Records
2. Add Student Record
3. Delete Student Record
4. Update Student Information
5. Show Instructor Details

6. Show Course Details with Enrolled Students
7. Show Course Details taken by Instructor
8. Deposit HRA To Salary
9. Deduct TDS From Salary
10. EXIT The Program

Enter your choice (1-10): 10

Exiting... Goodbye!

Process finished with exit code 0